



202X

操作系统寒假帮扶讲座1

2024.2.21

特别提醒：此PPT受众主要为补考同学，对于参加正常考试的同学仅供参考！

第1章 计算机系统概述

取指-执行

- 计算机部件：

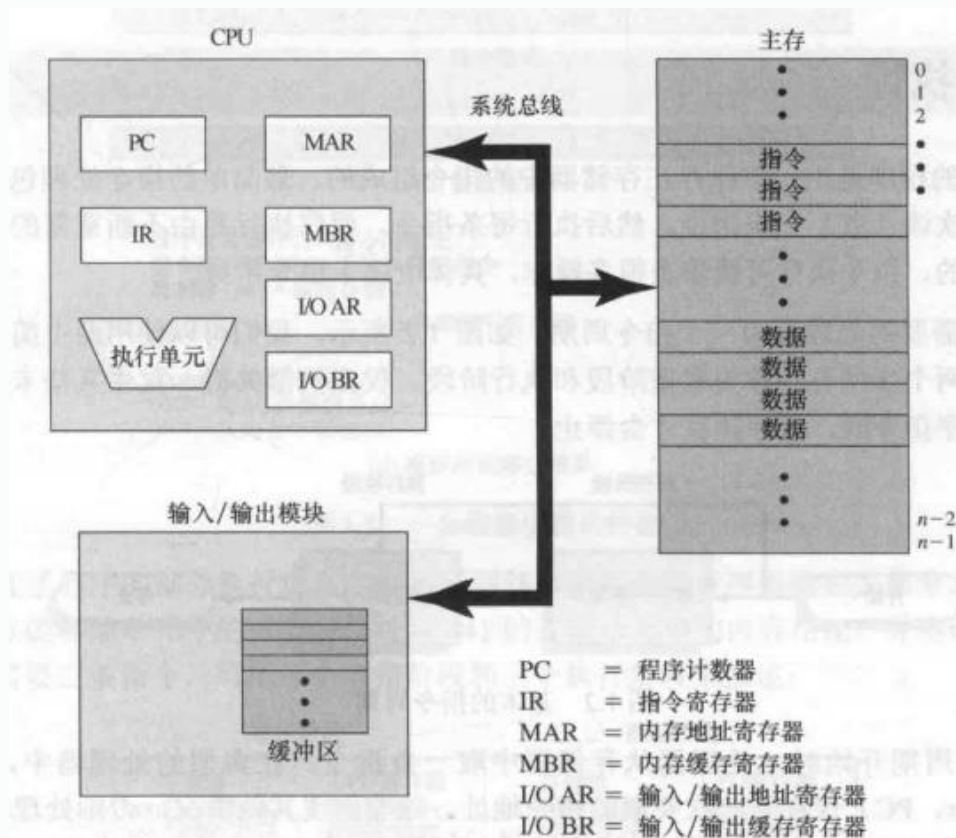


图 1.1 计算机部件：顶视图

第1章 计算机系统概述

• 中断：

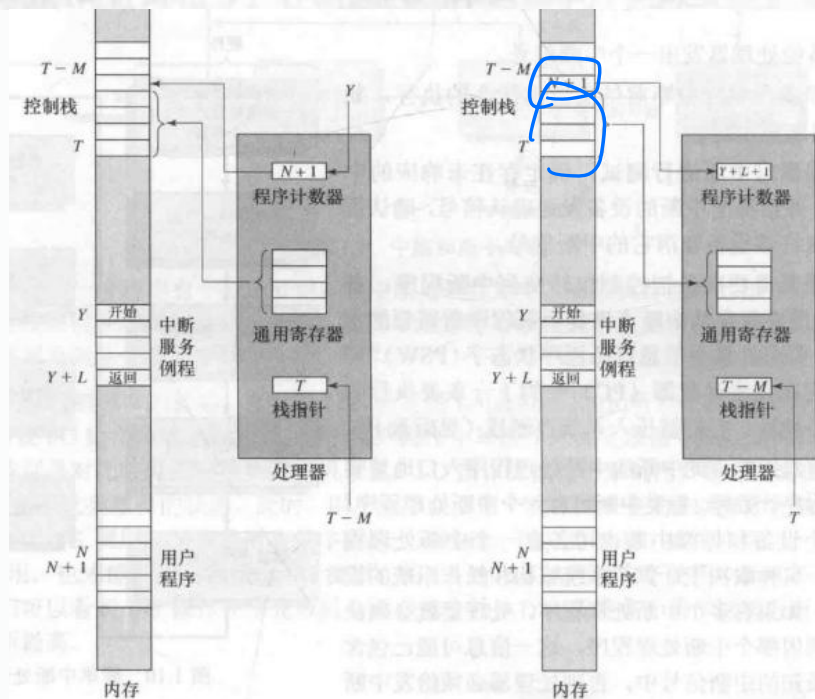
表 1.1 中断的分类

程序中断	在某些条件下由指令执行的结果产生，如算术溢出、除数为 0、试图执行一条非法机器指令及访问用户不允许的存储器位置
时钟中断	由处理器内部的计时器产生，允许操作系统以一定的规律执行函数
I/O 中断	由 I/O 控制器产生，用于发信号通知一个操作的正常完成或各种错误条件
硬件失效中断	由诸如掉电或存储器奇偶校验错之类的故障产生

• 中断处理：



图 1.10 简单中断处理



(a) 在存储单元N中的指令之后发生中断

(b) 从中断返回



第2章 操作系统概述

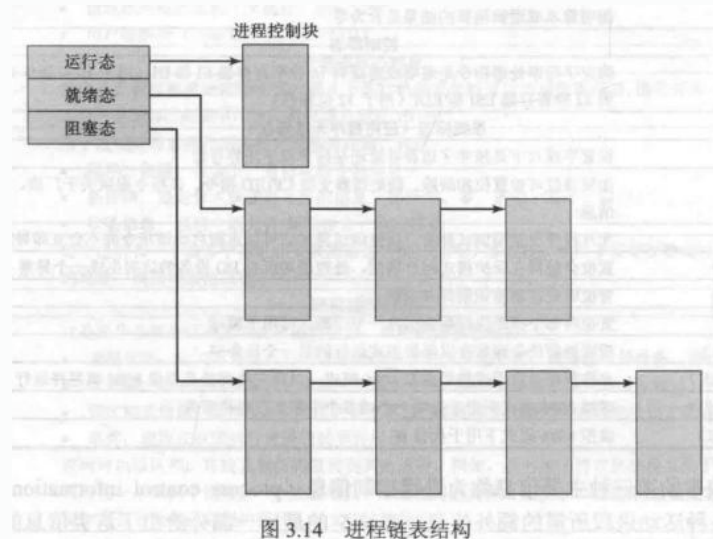
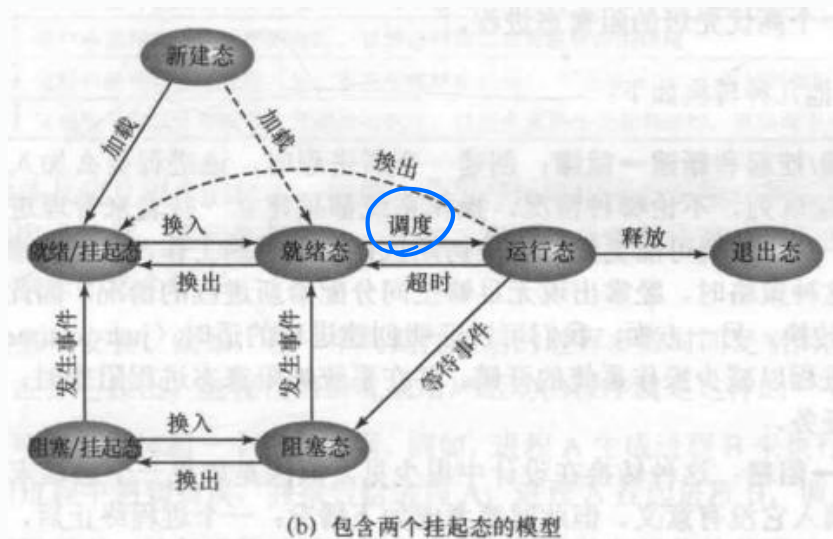
- 操作系统：**控制**应用程序执行的程序，是应用程序和计算机硬件间的**接口**
- 功能：用户/计算机接口、资源管理器
- 操作系统开发中的主要成就：进程、信息保护和安全、内存管理、调度和资源管理

第3章 进程描述和控制

- 进程：一个**正在执行的程序**/一个**正在**计算机上执行的**程序实例**/能分配给处理器并由处理器执行的实体/由一组执行的指令、一个当前状态和一组相关的系统资源**表征的活动单元**
- 进程的组成：程序代码+相关数据+进程控制块 *pcb*
- 进程控制块：由操作系统创建和管理，包含了操作系统所需进程的所有信息
 - ①标识符：与进程相关的**唯一标识符**，用来区分其他进程
 - ②状态：就绪态、运行态、阻塞态等
 - ③优先级：相对于其他进程的优先顺序
 - ④程序计数器：程序中即将执行的下一条指令的地址
 - ⑤内存指针：包含程序代码和相关数据的指针，以及与其他进程共享内存块的指针
 - ⑥上下文数据：进程执行时处理器的寄存器中的数据
 - ⑦I/O状态信息：包括显式I/O请求、分配给进程的I/O设备和被进程使用的文件列表等
 - ⑧记账信息：包括处理器时间总和、使用的时钟数总和、时间限制、记账号等

第3章 进程描述和控制

- 进程状态：



第3章 进程描述和控制

- 操作系统控制表：操作系统管理进程和资源，掌握每个进程和资源的当前状态
- 进程映像：

表 3.4 进程映像中的典型元素

项 目	说 明
用户数据	用户空间中的可修改部分，包括程序数据、用户栈区域和可修改的程序
用户程序	待执行的程序
栈	每个进程有一个或多个后进先出（LIFO）栈，栈用于保存参数、过程调用地址和系统调用地址
进程控制块	操作系统控制进程所需的数据（见表 3.5）

- 进程映像在虚存中的结构：

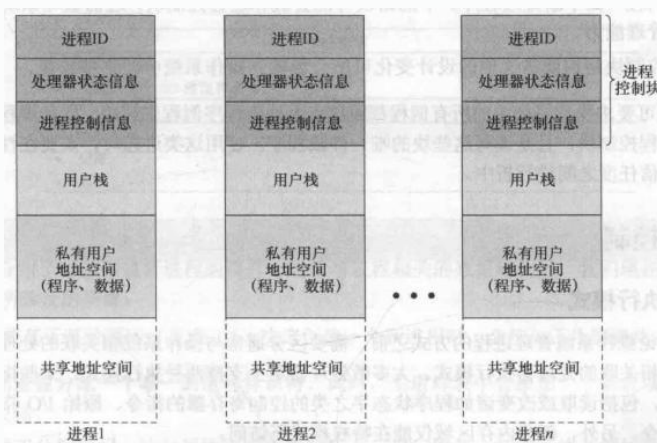


图 3.13 虚存中的用户进程

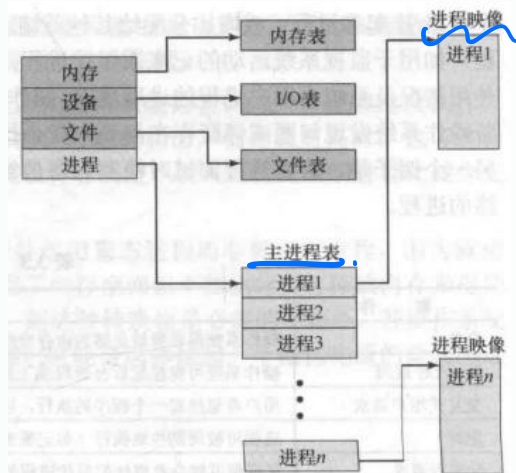


图 3.11 操作系统控制表的通用结构

第3章 进程描述和控制

- 执行模式：

- ①用户模式：**用户程序**通常在该模式下运行

- ②内核/系统/控制/特权模式：可以**读取或改变**如PSW之类的**控制寄存器的指令**、**原始I/O指令**和与**内存管理**相关的指令，部分内存区域仅能在该模式下访问，保护操作系统和重要的操作系统表不受用户程序的干扰

- 如何确定处理器正在什么模式下运行：2位RPL字段，¹⁰**级别0**为最高特权级别，¹¹**级别3**为最低特权级别，内核模式使用级别0，用户模式使用其他级别

- 进程创建：

- ①为新进程分配一个唯一的进程标识符

- ②为进程分配空间：包括进程映像中的所有元素

- ③初始化进程控制块

- ④设置正确的链接

- ⑤创建或扩充其他数据结构

第3章 进程描述和控制

- 何时切换进程：可在操作系统从当前正运行进程中获得控制权的**任何时刻**发生

①中断：控制权首先交给中断处理器，完成一些基本工作后，再将控制权转给相关的操作系统例程。如时钟中断、I/O中断、内存失效

②陷阱

③系统调用

↓
进程切换就绪态，并调用新进程

- 进程切换步骤：

①保存处理器的上下文，包括程序计数器和其他寄存器

②更新当前处于运行态进程的进程控制块

③把该进程的进程控制块移到相应的队列

④选择另一个进程执行（调度算法）

⑤更新所选进程的进程控制块，包括把进程的状态改为运行态

⑥更新内存管理数据结构

⑦载入程序计数器和其他寄存器先前的值，恢复处理器的上下文

第4章 线程

- 多线程：操作系统在**单个进程**内支持**多个并发**执行路径的能力
- 线程与进程的区别：

①单线程进程模型：**进程**的表示包括：PCB、用户地址空间、用户栈、内核栈

②多线程进程模型：进程的表示包括：PCB、用户地址空间

线程的表示包括：单独的线程控制块、单独的栈

进程中所有线程**共享**该进程的状态和资源，所有线程都驻留在同一块地址空间中，可以访问相同的数据

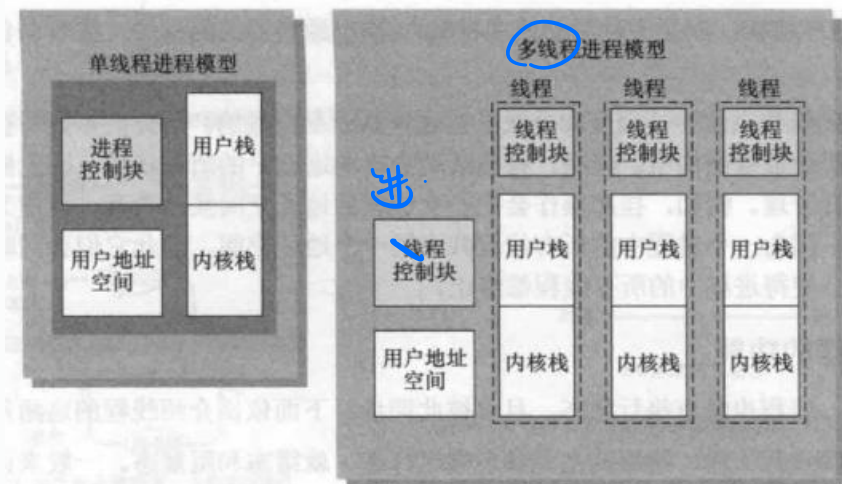


图 4.2 单线程和多线程进程模型

第5章 并发：互斥和同步

- 关键术语：

表 5.1 与并发相关的关键术语

<u>原子操作</u>	一个函数或动作由一个或多个指令的序列实现，对外是不可见的；也就是说，没有其他进程可以看到其中间状态或能中断此操作。要保证指令序列要么作为一个组来执行，要么都不执行，对系统状态没有可见的影响。原子性保证了并发进程的隔离
<u>临界区</u>	一段代码，在这段代码中进程将访问共享资源，当另外一个进程已在这段代码中运行时，这个进程就不能在这段代码中执行
<u>死锁</u>	两个或两个以上的进程因每个进程都在等待其他进程做完某些事情而不能继续执行的情形
<u>活锁</u>	两个或两个以上的进程为响应其他进程中的变化而持续改变自己的状态但不做有用的工作的情形
<u>互斥</u>	<u>当一个进程在临界区访问共享资源时，其他进程不能进入该临界区访问任何共享资源的情形</u>
<u>竞争条件</u>	多个线程或进程在读写一个共享数据时，结果依赖于它们执行的相对时间的情形
<u>饥饿</u>	一个可运行进程尽管能继续执行，但被调度程序无限期地忽视，而不能被调度执行的情形

- 举例：

```
void echo()  
{  
    chin = getchar();  
    chout = chin;  
    putchar(chout);  
}
```

P1
chin = x
↓
y

P2.
chin = y
↓
y

- 问题本质：共享全局变量chin

- 说明：若需要保护共享的全局变量（以及其他共享的全局资源），唯一的办法是控制访问该变量的代码，即一次只允许一个进程进入echo，并且只有在echo运行结束后，才允许另一个进程进入

第5章 并发：互斥和同步

- Dijkstra设计的基本原理：两个或多个进程可以通过简单的信号进行合作，可以强迫一个进程在某个位置停止，直到它接收到一个特定的信号。任何复杂的合作需求都可通过适当的信号结构得到满足。为了发信号，使用一个成为信号量的特殊变量。
- 发信号：semSignal(s) 接收信号：semWait(s)
- semWait 使信号量-1，若值变为负数，则阻塞执行semWait的进程，否则进程继续执行
semSignal 使信号量+1，若值小于等于0，则被semWait操作阻塞的进程解除阻塞
- 信号量初始化成非负数，代表着发出semWait操作后可以立即继续执行的进程的数量
- 解决互斥：

```
/* program mutual exclusion */
const int n = /* 进程数 */;
semaphore s = 1;
void P(int i)
{
    while (true) {
        semWait(s);
        /* 临界区 */;
        semSignal(s);
        /* 其余部分 */;
    }
}
void main()
{
    parbegin (P(1), P(2), ..., P(n));
}
```

图中包含以下手写标注：

- “申请”：指向 `semWait(s);` 语句。
- “临界区”：圈出 `/* 临界区 */;` 语句。
- “释放”：指向 `semSignal(s);` 语句。

第5章 并发：互斥和同步

$P_1 \rightarrow P_2$
semSignal, semWait
bf · mutex = 1

- 生产者/消费者问题：有一个或多个生产者生产某种类型的数据，并放置在缓冲区中；有一个消费者从缓冲区中取数据，每次取一项；系统保证避免对缓冲区的重复操作，即在任何时候只有一个主体（生产者或消费者）可访问缓冲区。确保当缓存已满时，生产者不会继续向其中添加数据；当缓存为空时，消费者不会从中移走数据。
① 缓存不满 → 生产者加数据。
② 缓存不满 → 消费者取。
full = 0
empty = n
- 补充：进程同步：进程的直接制约关系；为了完成某种任务而建立的2个或多个进程，这些进程因为需要在某些位置上协调它们的工作次序而等待。

semaphore mutex = 1, full = 0, empty = n;

```
void Producer() {  
    while(1) {  
        生产;  
        semWait(empty);  
        semWait(mutex);  
        放入缓冲区;  
        semSignal(mutex);  
        semSignal(full);  
    }  
}
```

```
void Consumer() {  
    while(1) {  
        semWait(full);  
        semWait(mutex);  
        取出一个产品;  
        semSignal(mutex);  
        semSignal(empty);  
        消费;  
    }  
}
```

第5章 并发：互斥和同步



- 例：有三个进程A、B和C合作解决文件打印问题：进程A将文件记录从磁盘读入缓冲区1，每执行一次读一个记录；进程B将缓冲区1的内容复制到缓冲区2，每执行一次复制一个记录；进程C将缓冲区2的内容打印出来，每执行一次打印一个记录；缓冲区1和2的大小都等于一个记录大小。请用信号量操作来保证文件的正确打印。

同步：1不满 → A取 $e1=1$ 1不满 → B取 $f1=0$ 2不满 → B取 $e2=1$ 2不满 → C取 $f2=0$

```
A() {  
    while(1) {  
        semWait(e1);  
        读入;  
        semSignal(f1);  
    }  
}
```

```
B() {  
    while(1) {  
        semWait(f1);  
        semWait(e2);  
        复制;  
        semSignal(e1);  
        semSignal(f2);  
    }  
}
```

```
C() {  
    while(1) {  
        semWait(f2);  
        打印;  
        semSignal(e2);  
    }  
}
```

第6章 并发：死锁和饥饿

- 死锁：一组相互**竞争**系统资源或进行通信的进程间的“**永久**”阻塞
- 当一组进程中的**每个进程**都在等待某个事件（如等待释放所请求的资源），而仅有这组进程中被**阻塞**的其他进程才可触发该事件
- 解决方法：**鸵鸟算法**



(a) 遇到危险的鸵鸟



(b) 假装什么都没看见



202X

操作系统寒假帮扶讲座2

2024.2.22

特别提醒：此PPT受众主要为补考同学，对于参加正常考试的同学仅供参考！

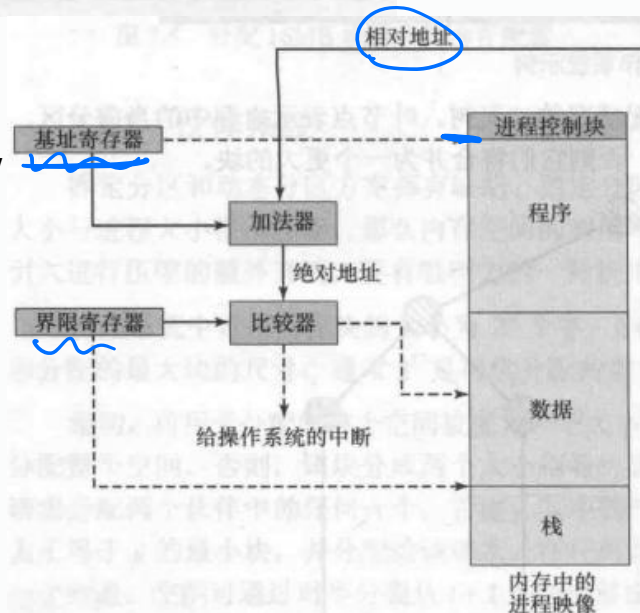
第7章 内存管理

- 术语：

表 7.1 内存管理术语

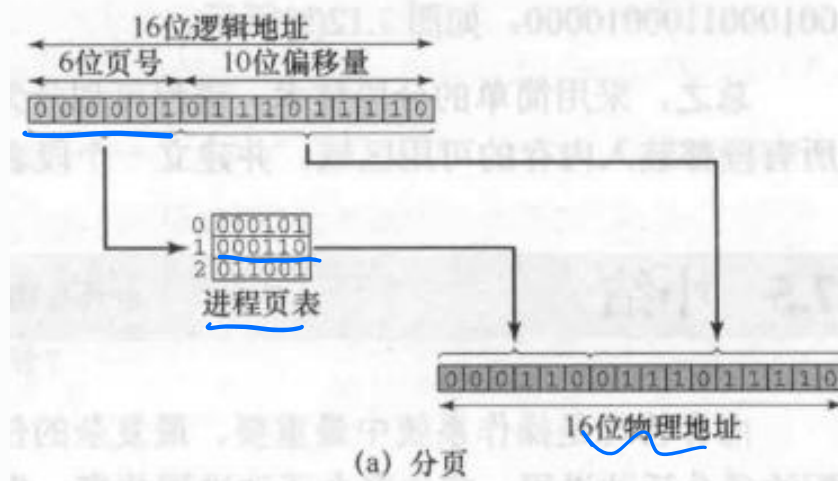
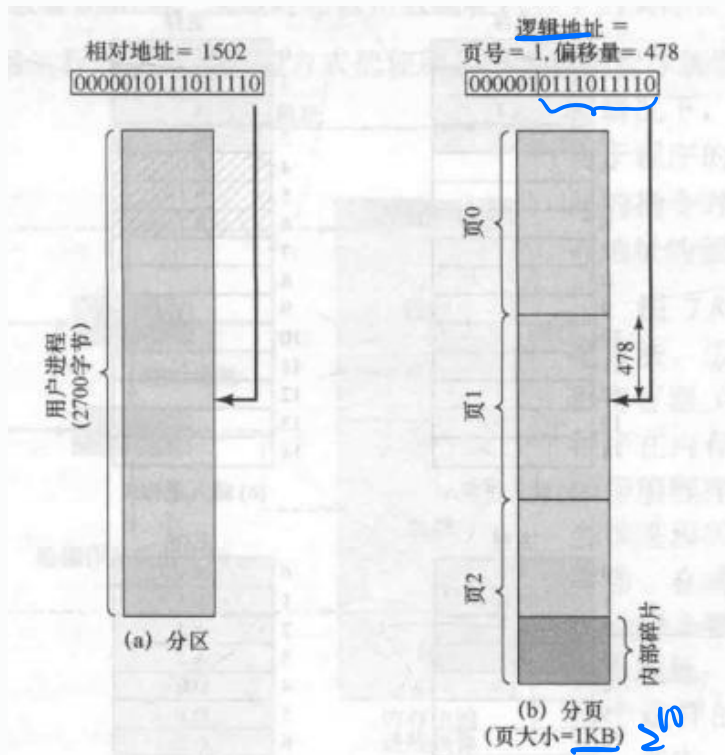
页框	内存中固定长度的块
页	固定长度的数据块，存储在二级存储器中（如磁盘）。数据页可以临时复制到内存的页框中
段	变长数据块，存储在二级存储器中。整个段可以临时复制到内存的一个可用区域中（分段），或可以将一个段分为许多页，然后将每页单独复制到内存中（分段与分页相结合）

- 内存分区：固定分区、动态分区、伙伴系统
- 逻辑地址：指与当前数据在内存中的物理分配地址无关的访问地址，在执行对内存的访问之前必须把它转换为物理地址
- 相对地址：逻辑地址的一个特例，相对于某些已知点（通常是程序的开始）的存储单元
- 物理地址：数据在内存中的实际位置
- 重定位：把使用相对地址的程序加载到内存，需要把相对地址转换为物理地址



第7章 内存管理

- 分页：内存被划分为大小固定、相等的块，且块相对比较小，每个进程也被分成同样大小的小块，则进程中称为页的块可以分配到内存中称为页框的可用块
- 页表：操作系统为每个进程维护一个页表，页表给出了该进程的每页所对应页框的位置
- 计算地址：



第7章 内存管理

- 分段：把程序和与其相关的数据划分到几个段中，段的长度可以不等
- 段表：每个进程都有一个段表，每个段表项给出相应段在内存中的起始地址和段的长度
- 计算地址：

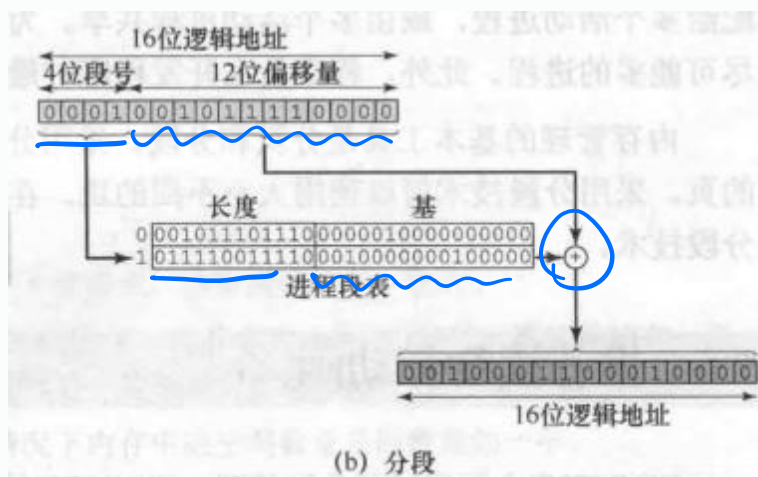
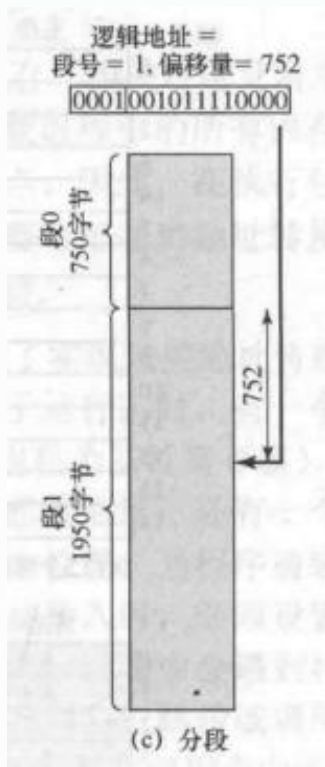


图 7.12 逻辑地址转换为物理地址示例

第8章 虚拟内存

- 术语：

表 8.1 虚拟内存术语

虚拟内存	在存储分配机制中，尽管备用内存是主存的一部分，但它也可被寻址。程序引用内存使用的地址与内存系统用于识别物理存储站点的地址是不同的，程序生成的地址会自动转换为机器地址。虚拟存储的大小受计算机系统寻址机制和可用的备用内存量的限制，而不受主存储位置实际数量的限制
虚拟地址	在虚拟内存中分配给某一位置的地址，它使得该位置可被访问，就好像是主内的一部分那样
虚拟地址空间	分配给进程的虚拟存储
地址空间	用于某进程的内存地址范围
实地址	内存中存储位置的地址

- 分页和分段特点：

表 8.2 分页和分段的特点

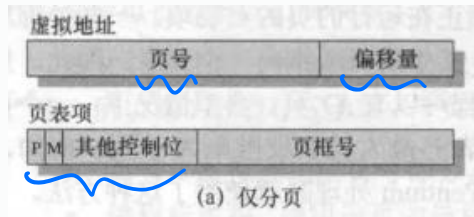
简单分页	虚存分页	简单分段	虚存分段
内存划分为大小固定的小块，称为页框	内存划分为大小固定的小块，称为页框	内存未划分	内存未划分
程序被编译器或内存管理系统划分为页	程序被编译器或内存管理系统划分为页	由程序员给编译器指定程序段（即由程序员决定）	由程序员给编译器指定程序段（即由程序员决定）
页框中有内部碎片	页框中有内部碎片	无内部碎片	无内部碎片
无外部碎片	没有外部碎片	有外部碎片	有外部碎片
操作系统须为每个进程维护一个页表，以说明每页对应的页框	操作系统须为每个进程维护一个页表，以说明每页对应的页框	操作系统须为每个进程维护一个段表，以说明每段中的加载地址和长度	操作系统须为每个进程维护一个段表，以说明每段中的加载地址和长度
操作系统须维护一个空闲页框列表	操作系统须维护一个空闲页框列表	操作系统须维护一个内存中的空闲空洞列表	操作系统须维护一个内存中的空闲空洞列表
处理器使用页号和偏移量来计算绝对地址	处理器使用页号和偏移量来计算绝对地址	处理器使用段号和偏移量来计算绝对地址	处理器使用段号和偏移量来计算绝对地址
进程运行时，它的所有页必须都在内存中，除非使用了覆盖技术	进程运行时，并非所有页都须在内存中，仅在需要时才读入页	进程运行时，其所有页都须在内存中，除非使用了覆盖技术	进程运行时，并非其所有段都须在内存中，仅在需要时才读入段
	把一页读入内存可能需要把另一页写出到磁盘		把一段读入内存可能需要把另外一段或几段写出到磁盘

第8章 虚拟内存

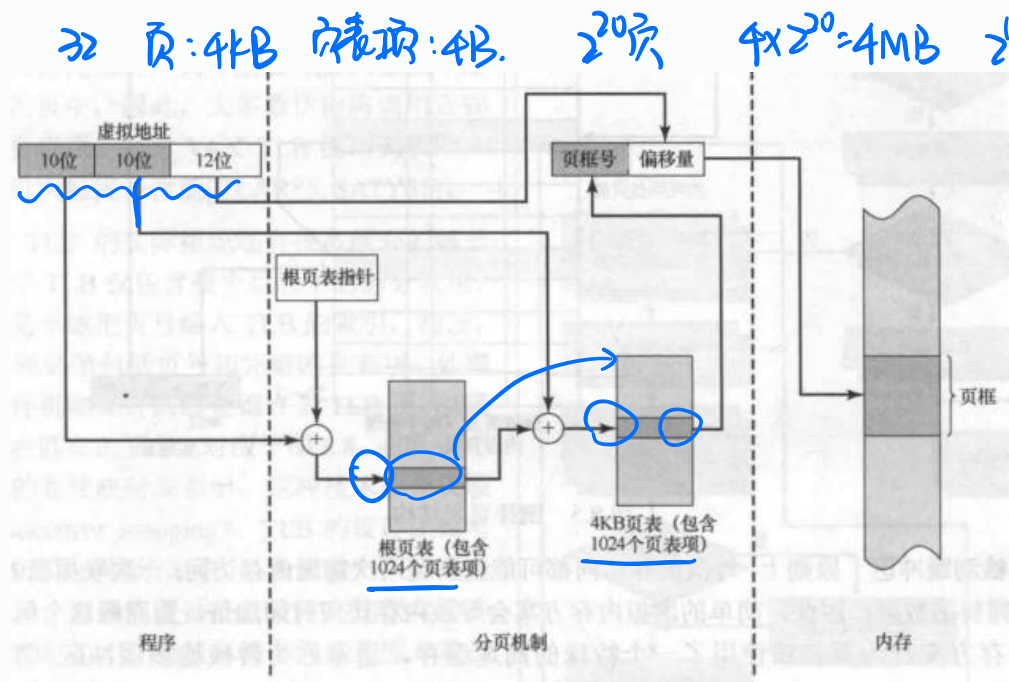
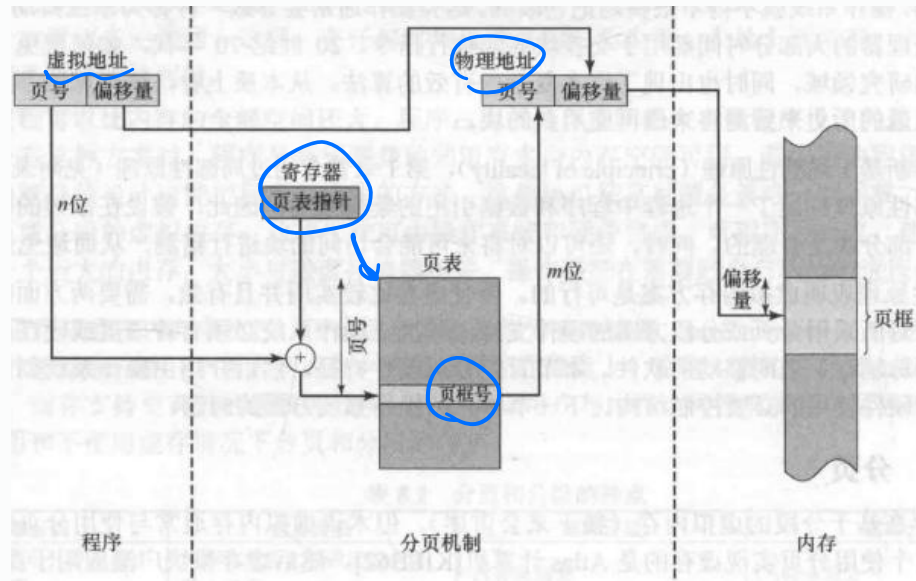
- 分页：

- 内存管理格式：

- 地址转换：



P：存在位、M：修改位



第8章 虚拟内存

- 分页：
 - TLB：保存最近用过的页表项

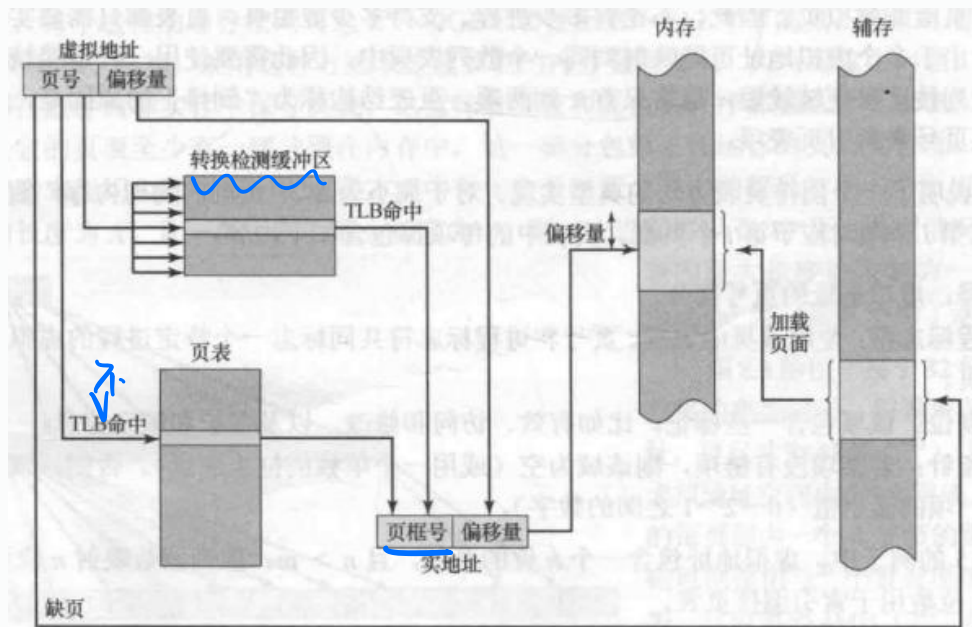


图 8.6 转换检测缓冲区的用法

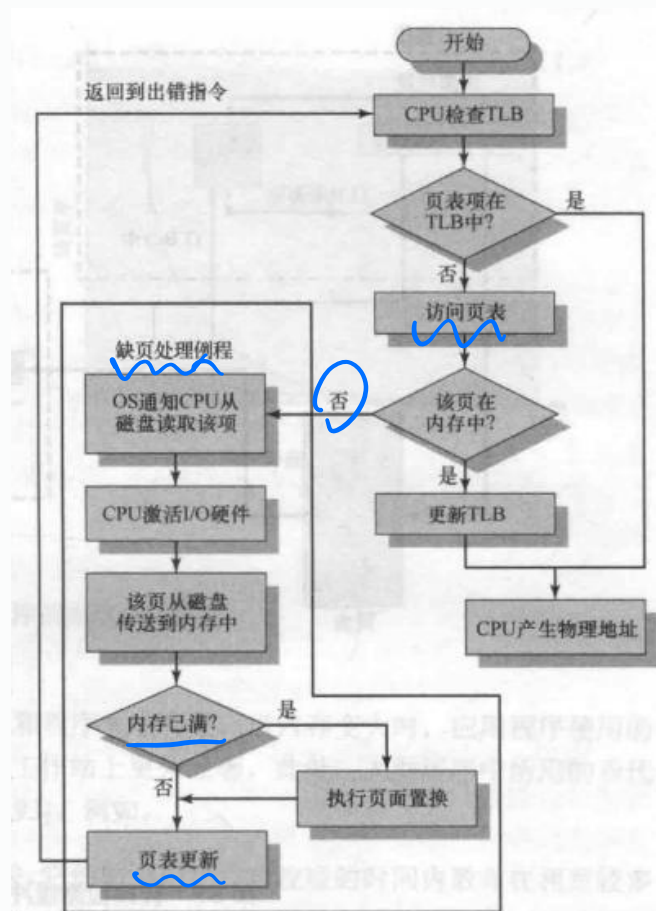


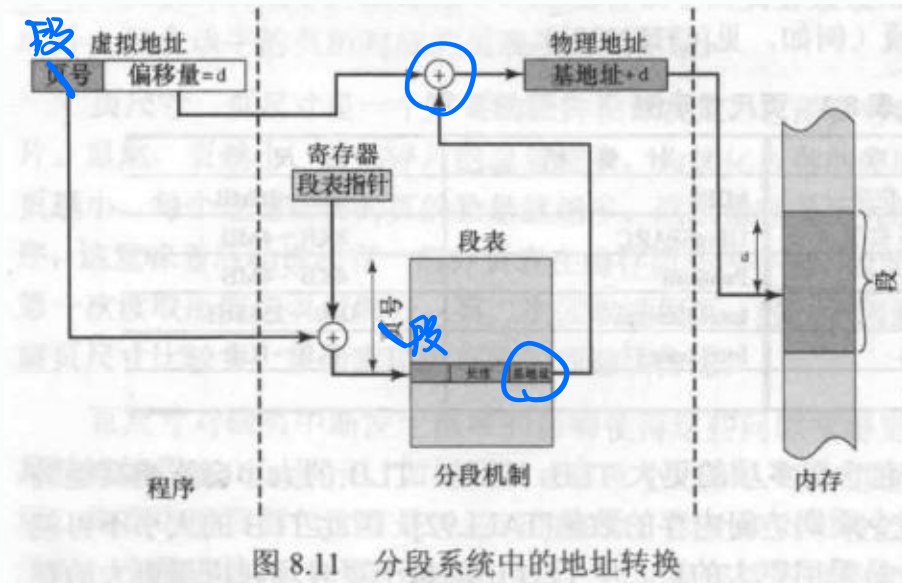
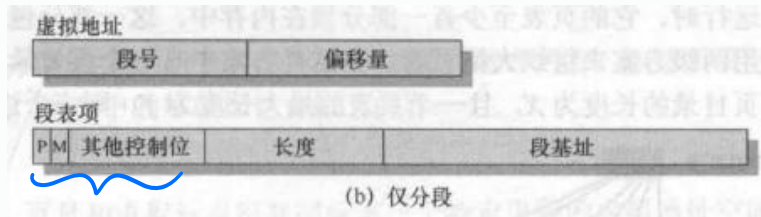
图 8.7 分页和转移检测缓冲区的操作

第8章 虚拟内存

- 分段：

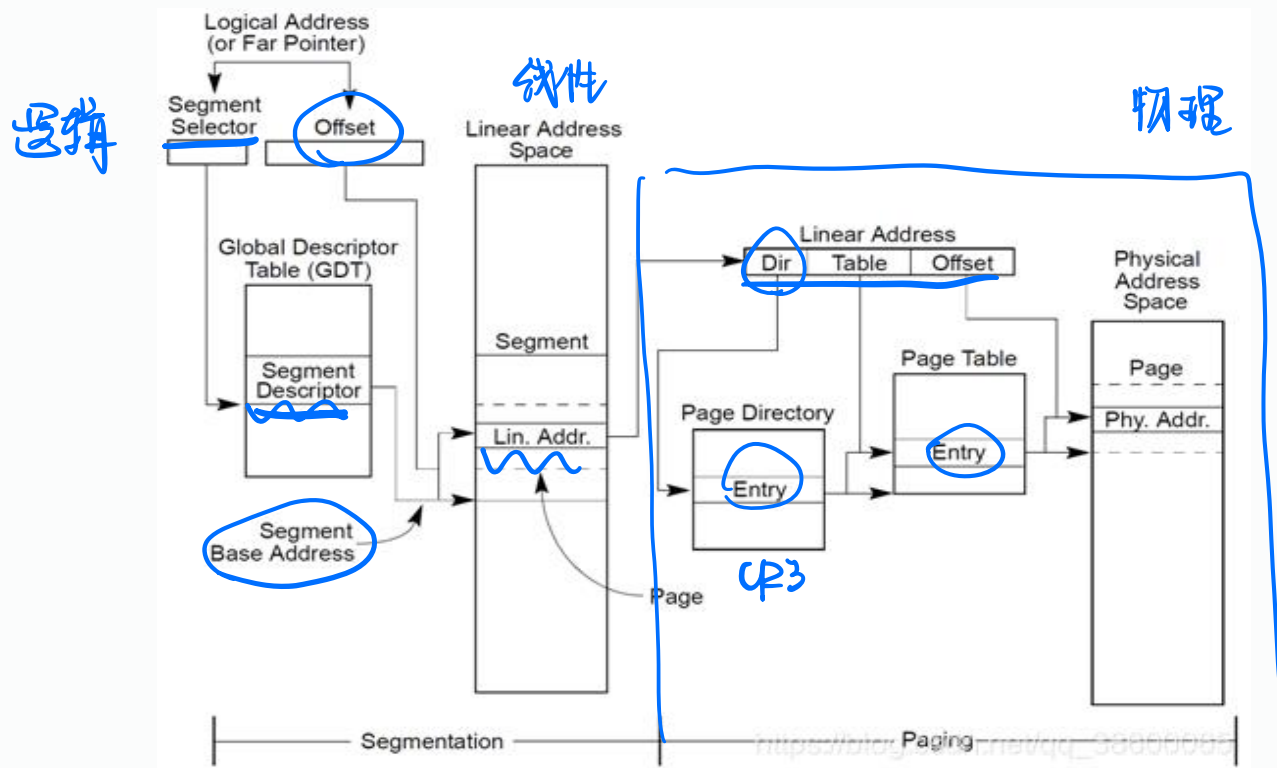
- 内存管理格式：

- 地址转换：



第8章 虚拟内存

- 段页式：用户的地址空间被程序员划分为许多段，每段依次划分为许多固定大小的页
- 地址转换：每个进程都使用一个段表和一些页表（多级）



第9章 单处理器调度

- 调度：决定了哪个进程须等待、哪个进程能继续运行。本质上属于队列管理问题，用于在排队环境中减少延迟并优化性能

- 调度算法：

①先来先服务FCFS：也称先进先出FIFO（非抢占）

②轮转RR：周期性产生时钟中断，出现中断时，当前正运行的进程放置到就绪队列中，基于FCFS选择下一个就绪队列运行（抢占）

③最短进程优先SPN：选择预计处理时间最短的进程（非抢占）

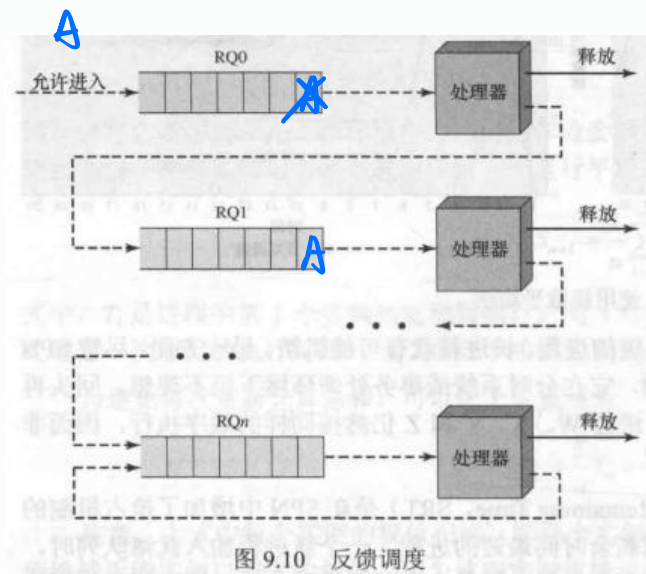
④最短剩余时间SRT：选择预期剩余时间最短的进程（抢占）

⑤最高响应比优先HRRN：选择响应比R最大的就绪进程（非抢占）

$$R = \frac{w + s}{s}$$

w 为等待处理器的时间， s 为预计服务的时间

⑥反馈法：调度基于抢占原则（按时间片）并使用动态优先机制



第9章 单处理器调度

q=2 AABBC EE ABBEE BEE.

A:8 C:5

B:13 E:15

(4/4)

- 例1: 考虑下面的进程, 在时刻0以1-4的顺序到达 (到达间隔可以忽略), 采用基于时间片轮转 (RR) 算法, 计算时间片 $q=1$ 和 $q=2$ 时平均周转时间

q=1 ABC E ABE AB E BE E.

A:8 E:15

B:13

C:3

(3/4)

进程名	到达顺序	执行时间
A	1	3
B	2	5
C	3	1
E	4	6

- 例2: 假设下列进程在所指定的时刻到达就绪队列等待执行, 每个进程将运行相应的区间时间。

①使用FCFS算法, 计算进程的平均周转时间; ②使用SPN算法, 计算进程的平均周转时间;

① 到 结

P1 0.0 8

P2 0.4 12

P3 1.0 14

P4 2.0 15

② 到 结

P1 0.0 8

P2 0.4 15

P3 1.0 11

P4 2.0 9

进程	到达时间	区间时间
P1	0.0	8
P2	0.4	4
P3	1.0	2
P4	2.0	1

$$\frac{1}{4}(8+11.6+13+13)=11.4$$

$$\frac{1}{4}(8+14.6+10+7)=9.9$$

第11章 I/O管理和磁盘调度

- 磁盘性能参数：

①寻道时间：将磁头臂移到指定磁道所需的时间

②旋转延迟：将磁盘的待访问地址区域旋转到读/写磁头可访问的位置所需的时间

③传输时间：向磁盘传送或从磁盘传送的时间，取决于磁盘的旋转速度。

总平均存取时间： $T_a = T_s + \frac{1}{2r} + \frac{b}{rN}$ ， b 表示要传送的字节数； N 表示一个磁道中的字节数； r 表示旋转速度

- 磁盘调度策略：磁盘请求队列

①先进先出FIFO：按顺序处理队列中的项目

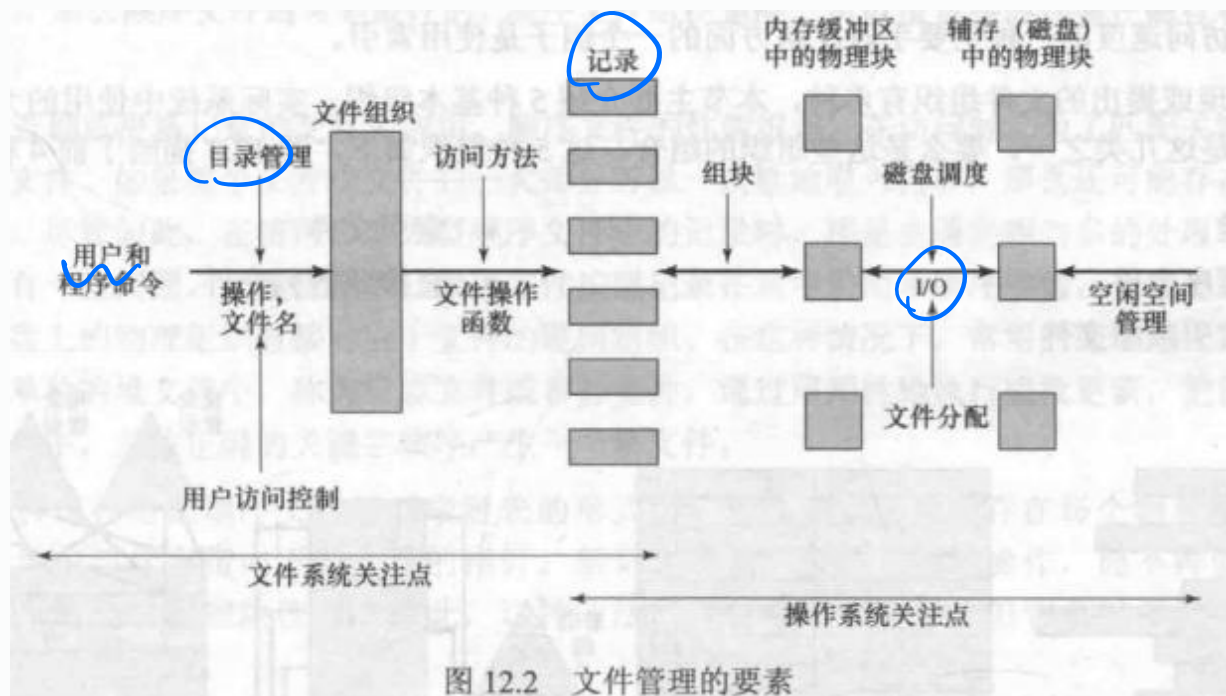
②最短服务时间优先SSTF：选择使磁头臂从当前位置开始移动最少的磁盘I/O请求

③SCAN（~~X~~电梯算法）：要求磁头臂仅沿一个方向移动，并在途中满足所有未完成的请求，直到（这个方向上的最后一个磁道，或者在）这个方向上没有其他请求为止。接着反转服务方向，沿相反方向扫描

④C-SCAN（循环SCAN）：扫描限定在一个方向上。当访问到沿某个方向的最后一个磁道时，磁头臂返回到磁盘相反方向末端的磁道，并再次开始扫描。

第12章 文件管理

- 文件管理的要素：



第12章 文件管理

- 文件目录：

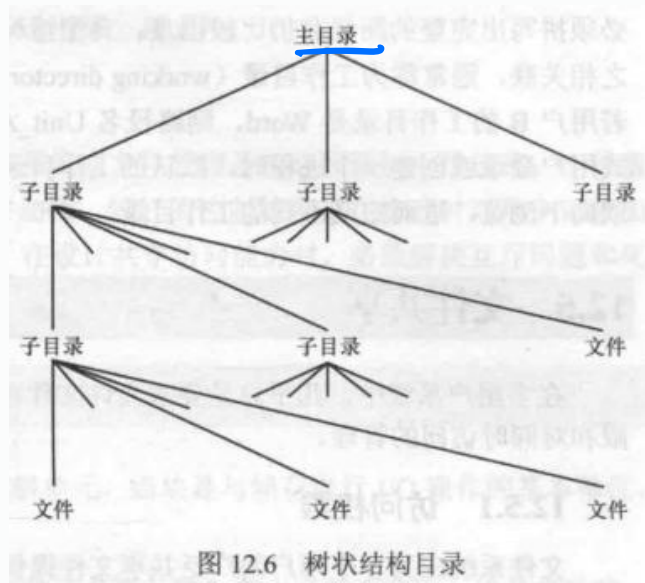


图 12.6 树状结构目录

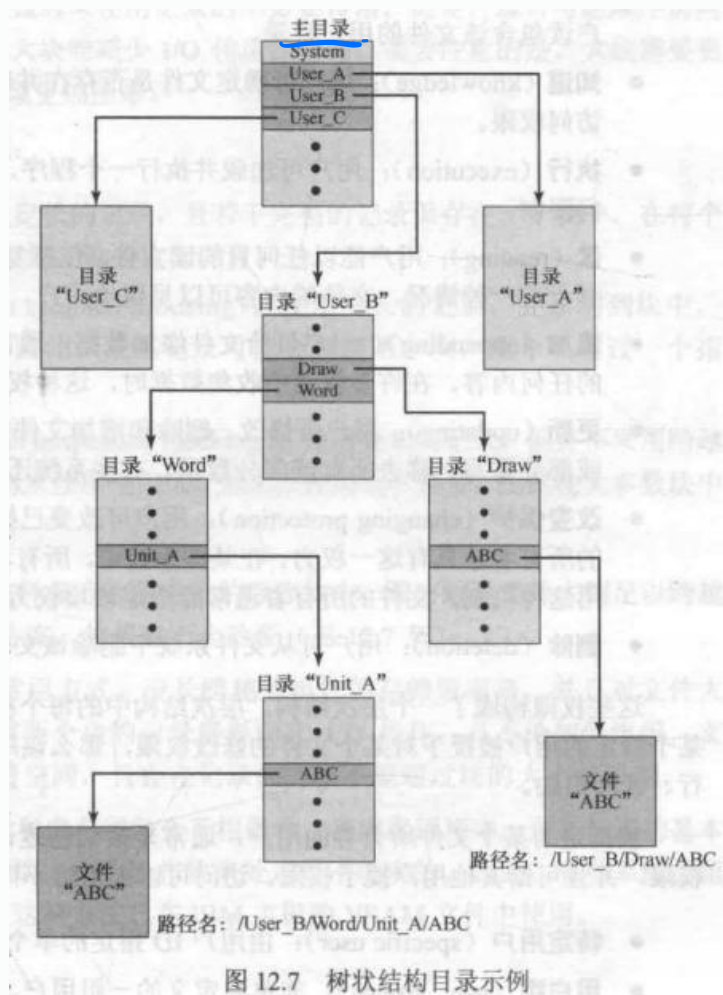


图 12.7 树状结构目录示例

第12章 文件管理

• 文件分配方法：

- ①连续分配：创建文件时，给文件分配一组连续的块
- ②链式分配：基于单个块，链中的每块都包含指向下一块的指针
- ③索引分配：每个文件在文件分配表中都有一个一级索引，分配给该文件的每个分区在索引中都有一个表项，文件的索引保存在一个单独的块中



图 12.9 连续文件分配



图 12.11 链式分配

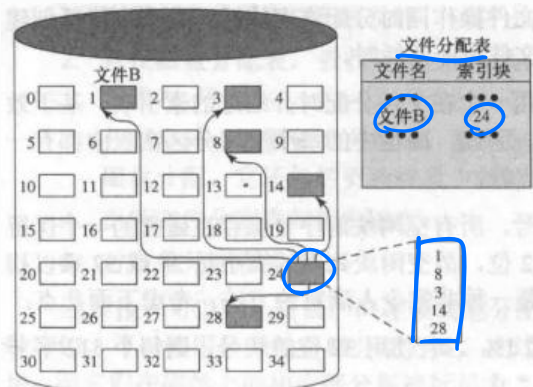
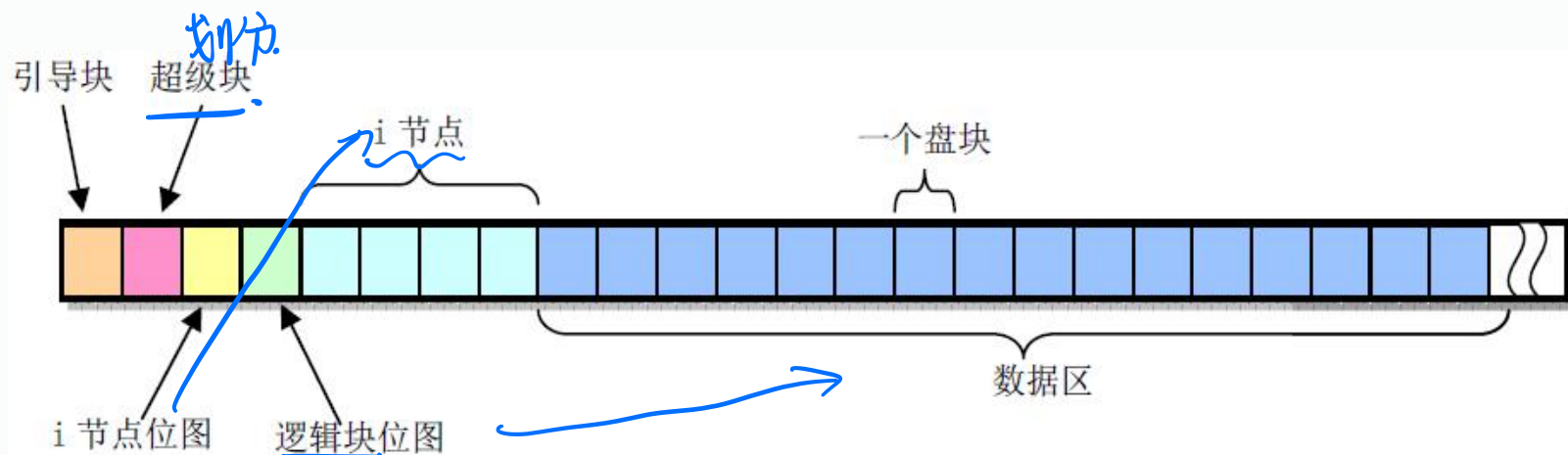


图 12.13 基于块的索引分配

第12章 文件管理

- Minix 文件系统：

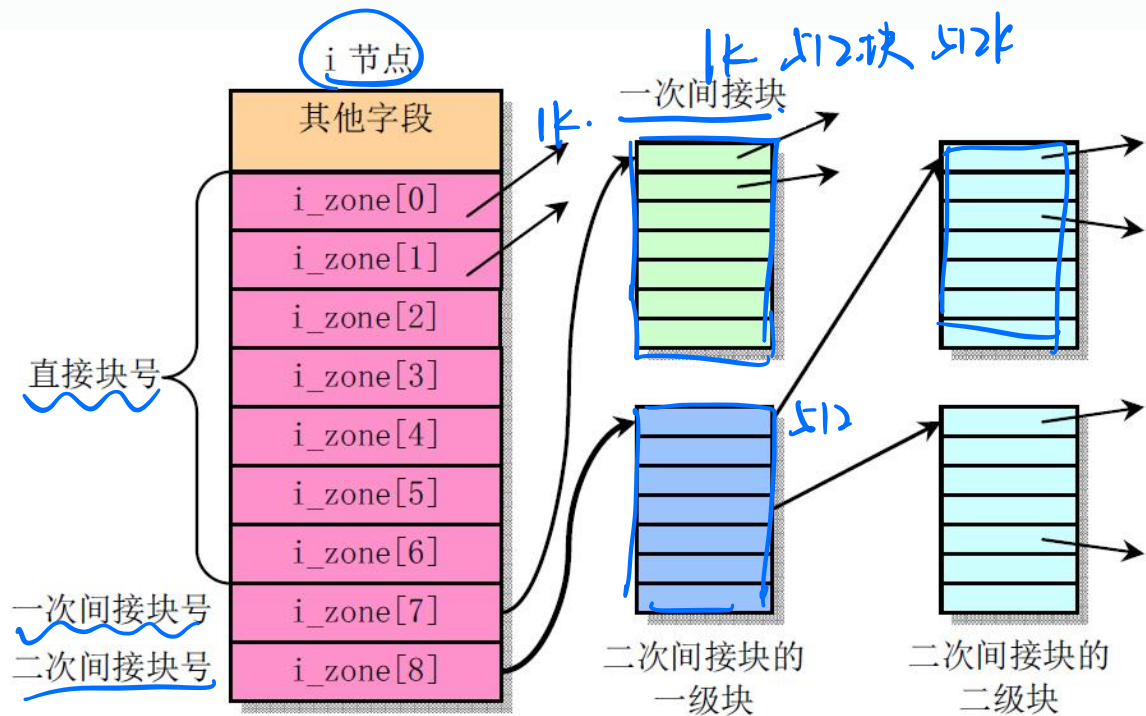


第12章 文件管理

7k + 512k + 512 × 512k.

- Minix 文件系统：

字段名称	数据类型	说明
<u>i_mode</u>	short	文件的类型和属性 (rwx 位)

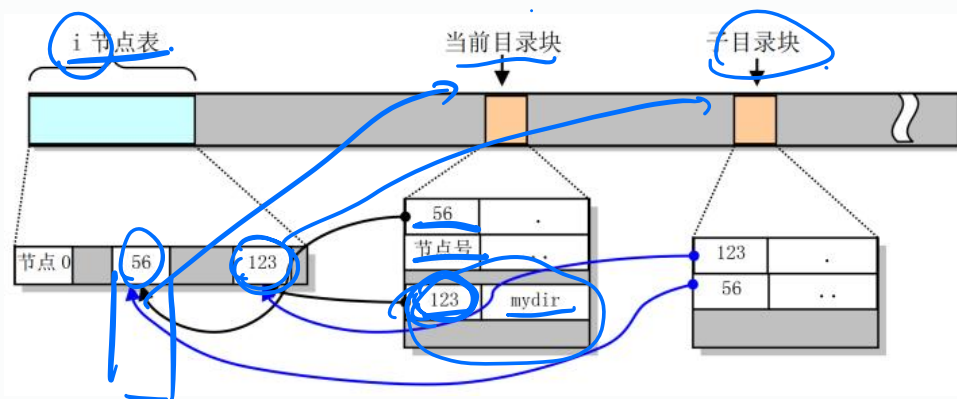


第12章 文件管理

• Minix 文件系统：

例1：如何访问文件 /usr/bin/文件名3？

1. 根目录文件的i节点位置是固定的，即第1号i节点
2. 1号i节点的数据块内容为根目录下的目录项列表
3. 通过该目录项列表匹配目录名 "usr"
4. 若找到，则可得到文件 "/usr"的i节点号i1
5. 根据i1号i节点的数据块，可以取得目录文件 "/usr"的内容，即子目录usr的文件目录项列表
6. 通过该目录项列表匹配目录名 "bin"
7. 若找到，则可得到文件 "/usr/bin"的i节点号i2
8. 根据i2号i节点的数据块，可以取得目录文件 "/usr/bin"的内容，即子目录bin的文件目录项列表
9. 通过该目录项列表匹配文件名 "文件名3"
10. 若找到，则可得到文件 "/usr/bin/文件名3"的i节点号i3
11. 根据i3号i节点的数据块，可以取得文件 "文件名3"的内容



第12章 文件管理

- Minix 文件系统：

例2：如何删除文件/usr/bin/vi？

1. 根目录文件的i节点位置是固定的，即第1号i节点
2. 1号i节点的数据块内容为根目录下的目录项列表
3. 通过该目录项列表匹配目录名 “usr”
4. 若找到，则可得到文件 “/usr”的i节点号i1
5. 根据i1号i节点的数据块，可以取得目录文件 “/usr”的内容，即子目录usr的文件目录项列表
6. 通过该目录项列表匹配目录名 “bin”
7. 若找到，则可得到文件 “/usr/bin”的i节点号i2
8. 根据i2号i节点的数据块，可以取得目录文件 “/usr/bin”的内容，即子目录bin的文件目录项列表
9. 通过该目录项列表匹配文件名 “vi”
10. 若找到，则可得到文件 “/usr/bin/vi”的i节点号i3
11. 找到i3节点，根据文件长度从i3节点中找到每个盘块的索引号，将i3节点位图和逻辑块位图中相应位置0（释放）

总结

- 假设当前操作系统采用**段页式**内存管理，并使用**二级页表**，页大小为**4K**，物理内存中已有多个进程，且实际物理内存的未分配空间为**14K**。现有一个编译好的程序test放在一个指定的路径下，其大小为**18K**，此程序的运行**至少需要两个时间片**。请结合你所学过的**进程管理**、**内存管理**、**文件管理**等操作系统知识，详细描述该程序从创建进程到运行结束，操作系统所做的主要操作有哪些。

- (1) 创建进程控制块 (PCB)：为新进程分配一个PCB，用于**存储进程的状态**、程序计数器、寄存器集合、内存管理信息（如**页目录表指针**（CR3存储页目录表的物理地址））、优先级、I/O状态信息等。
- (2) 分配内存空间：由于**程序大小为18K**，它将需要一个4K的**页目录表**和一个4K的**页表**。
- (3) 定位程序文件：查询文件分配表，得到根目录所在盘块，依次查询下一级目录所在盘块，最后查询到test程序文件所在的盘块。
- (4) 加载程序到内存：操作系统将test程序**代码和数据**从磁盘加载到分配给内存中（一页）。
- (5) 初始化进程上下文：设置程序计数器指向程序的起始地址，初始化所需的寄存器。
- (6) 执行程序：操作系统将进程状态设置为**就绪态**，等待调度运行。
- (7) 执行程序时发生**缺页中断**：当test数据或代码对应页不在物理内存中时，触发**缺页中断**，从用户态转到内核态，操作系统执行相应的中断处理程序，若内存不足，操作系统使用**页面置换算法**决定哪个页面被换出，由于需要I/O，**该进程被阻塞**，直到完成I/O转为就绪态。
- (8) 执行程序时发生**时钟中断**：由于该运行该程序至少需要2个时间片，第一个时间片耗尽时，执行时钟中断，**该程序转为就绪态**，CPU转而执行其他进程。
- (9) 运行结束：更新进程控制块，标记进程为退出态。与作业相关的表和其他信息会临时被操作系统保留，辅助或支持程序提取所需的信息，然后操作系统就不再保留任何与该进程相关的数据，从系统中删除该进程。